



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 07

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 320341704

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 14/10/25

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. **Actividades realizadas.** Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

1.- Agregarle a su propio coche (texturizado con la jerarquía de llantas y cofre) la luz de 1 faro frontal de color azul y posicionar a que ilumine hacia adelante y se mueva con el coche

Declaramos los modelos a utilizar para importar el carro

```
//Carro
Model Llanta_I;
Model Llanta_D;
Model Carro_M;
```

Después importamos el carro usando los .obj mismos que se crearon en la Practica 6

```
//Carro
Carro_M = Model();
Carro_M.LoadModel("Models/carro_texturas.obj");
Llanta_D = Model();
Llanta_D.LoadModel("Models/llanta_derecha.obj");
Llanta_I = Model();
Llanta_I.LoadModel("Models/llanta_izquierda.obj");
```

Para crear la luz azul de faro lo que hacemos es crear un SpotLight nuevo el cual tiene esos valores

```
// Faro color AZUL
spotLights[2] = SpotLight(
    0.0f, 0.0f, 1.0f,    // color
    1.0f, 2.0f,         // intensidad
    4.7f, 1.35f, 6.4f,  // posición inicial (se actualiza cada frame)
    1.0f, 0.0f, 0.0f,   // dirección inicial (hacia adelante del carro)
    1.0f, 0.0f, 0.0f,   // atenuación
    22.0f               // tamaño de iluminacion
);
spotLightCount++;
```

Tal y como se ve en los comentarios agregados cada valor representa algo distinto y estos valores son los iniciales es decir los que aparecerán por defecto al correr el código

```
//luz faro del carro
glm::vec3 faroPos(4.7f + mainWindow.getmuevex(), 1.35f, 6.4f);
glm::vec3 faroDir(1.0f, 0.0f, -0.05f);
spotLights[2].SetFlash(faroPos, faroDir);
```

Ahora para que la luz del carro sea dinámica es decir cambie sus valores tenemos dos vectores

faroPos() recibe la posición que va a tener la luz es decir en donde se debe de posicionar, la ubicación básicamente.

faroDir() recibe la dirección a la que debe de apuntar es decir hacia donde debe de iluminar

Estos van a pasarse para actualizar en el spotlight 2 (posición y dirección) para este frame, así, cuando después llamamos a SetSpotLights(), el shader recibe la luz ya pegada al faro.

Ahora en faroPos() recibe una función llamada getmuevex como valor, esta misma es usada para el carro que lo que hace es que tenga movimiento y se le agrega para que actualice su valor al momento de mover el carro así ambos se van actualizando al mismo tiempo.

```

//Instancia del coche
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getmuvex(), 1.8f, 8.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Carro_M.RenderModel();

//Llanta delantera izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(4.3f, -1.5f, 2.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
color = glm::vec3(0.5f, 0.5f, 0.5f); //Llanta con color gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_I.RenderModel();

//Llanta trasera izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(-3.5f, -1.5f, 2.2f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_I.RenderModel();

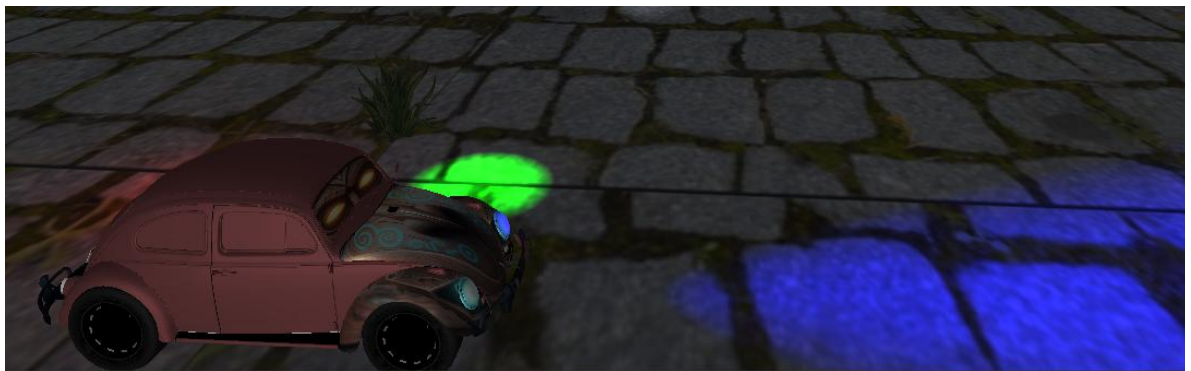
//Llanta delantera derecha
model = modelaux;
model = glm::translate(model, glm::vec3(4.3f, -1.5f, -2.1f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_D.RenderModel();

//Llanta trasera derecha
model = modelaux;
model = glm::translate(model, glm::vec3(-3.5f, -1.5f, -2.1f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_D.RenderModel();

```

Por último, agregamos nuestro carro con jerarquía para que al momento de mover con la función al modelo de carro se muevan todos juntos

Adelantándome un poco a la ejecución del código



Se puede observar una ligera separación de la luz y el carro y esto es producto del valor de la dirección, esto se puede arreglar si le pasamos un

valor negativo en el valor de “y” del vector, pero esto ocasionaría que la luz se vea como si estuviera solo en una parte del faro y no estuviera completo



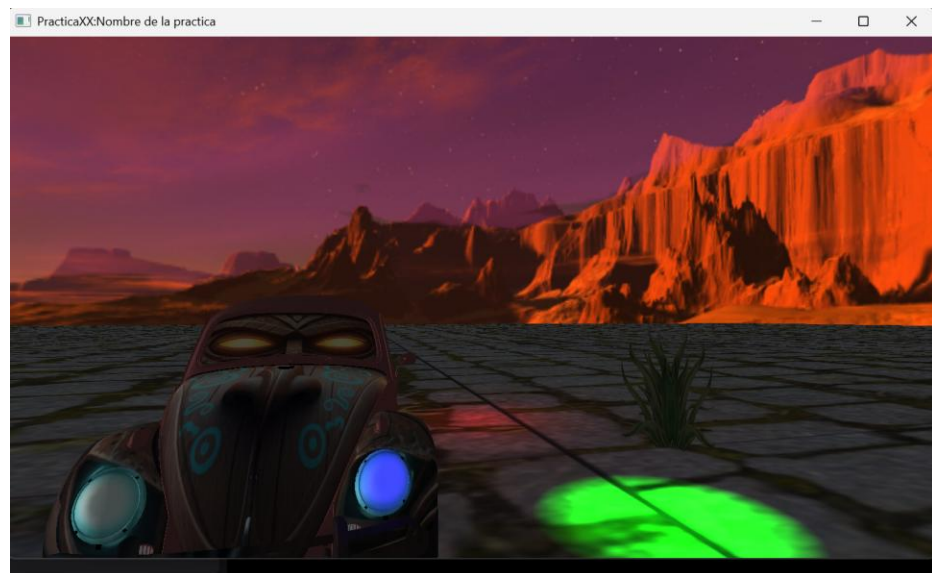
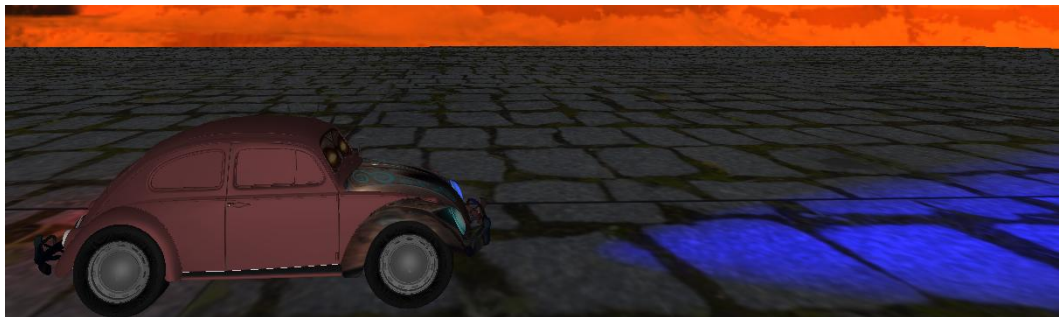
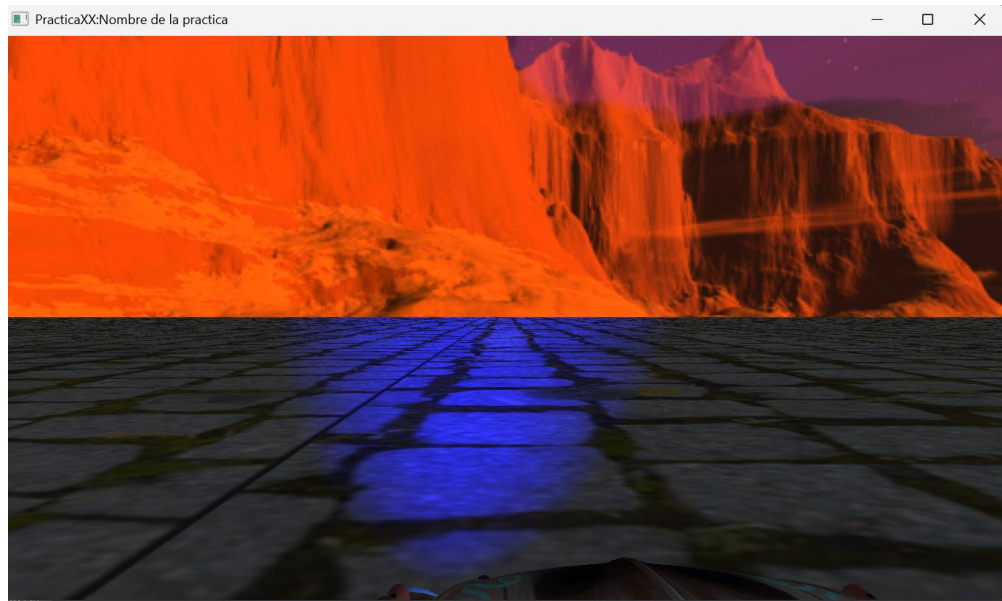
Aquí por ejemplo se redujo bastante la distancia pasandole un valor negativo es decir que se ubique más abajo, como si le estuviéramos dando una pequeña inclinación.

```
//luz faro del carro  
glm::vec3 faroPos(4.7f + mainWindow.getmuevex(), 1.35f, 6.4f);  
glm::vec3 faroDir(1.0f, -0.5f, -0.05f);  
spotLights[2].SetFlash(faroPos, faroDir);
```

Pero como se ve en la imagen esto provoca que se vea como si el faro solo tuviera luz en una parte.

Ahora otra cosa que se puede realizar es aumentarle el valor en el tamaño de iluminación y de igual forma se reduce la distancia, pero el problema es que ahora abarca mucho más espacio y no se ve muy natural que digamos, entonces por eso opte por dejarlo en un valor de 0 en “y” y decir que la luz apunte hacia “x” esto para que este frente al carro.

Ejecución



2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código

Realmente no sé si sea problema, pero igual que en la practica 6 cuando importe el carro en el código las coordenadas que había utilizado no funcionaban igual y tuve que acomodarlo nuevamente.

3. Conclusión:

Considero que la conclusión estuvo algo confusa, al inicio no me quedo del todo claro cómo funcionaba cada cosa, si bien no fue un ejercicio tan complicado si me costó trabajo ver cómo resolverlo porque no sabía exactamente como hacer uso de los spotLigths.

Me agrada que le vayamos agregando cosas de manera secuencial.